Lab 1 – Characterisation of the Data-Link Protocol Efficiency

Luís Kong (up202409115) · André Pereira (up202405143)
Class 3LEIC15 · Redes de Computadores (RC), L.EIC – FEUP, 2026/2027

Summary

We measured the efficiency S of our Stop-and-Wait ARQ data-link protocol in 198 file transfers over a virtual serial cable, varying the baud rate (C), the propagation delay (T_{prop}), the frame error rate (FER) and the frame size (L) one at a time. Without errors or delay, the protocol reaches S = 0.779, the limit set by start and stop bits and frame overhead. The theoretical formula $S = (1 - FER)/(1 + 2a)$ predicts every trend but overestimates S by a factor of 1.28; a corrected model that counts start and stop bits, overhead and replies matches all error-free points above 2400 bit/s to within 0.001. Two effects fall outside both models: the fixed 4 s timer halves S at 2400 bit/s and below, and BCC2 missed the errors in 9 of 84 distinct noisy transfers.

Introduction

The first lab implements reliable file transfer between two computers over an RS-232 serial link. The data-link layer provides framing with byte stuffing, error detection with a header check (BCC1) and a data check (BCC2, the XOR of the data bytes), acknowledgements and retransmission with Stop-and-Wait ARQ. The application layer splits the file into packets and uses only the data-link interface (`llOpenTx`, `llOpenRx`, `llSend`, `llReceive`, `llCloseTx`, `llCloseRx`), which keeps the two layers independent.

Three rules of our protocol shape its efficiency. The receiver answers a correct information (I) frame with RR and a frame with a bad BCC2 with REJ, which the transmitter answers at once by resending the frame. A frame with a bad header, or a lost reply, gets no answer, so the transmitter resends it only after a 4 s timeout, and gives up after 3 such retransmissions. Every frame waits for its reply before the next is sent.

This report evaluates the efficiency S = R/C, where R is the achieved bit rate, and compares it with the theoretical efficiency of Stop-and-Wait [1, slides 16, 34 and 36]:

$$S = \frac{1 - \mathrm{FER}}{1 + 2a}, \quad a = \frac{T_{\mathrm{prop}}}{T_f}, \quad T_f = \frac{8L}{C}, \quad \mathrm{FER} \approx 1 - (1 - \mathrm{BER})^{8L} \tag{1}$$

where L is the frame size in bytes.

Methodology

Setup. All transfers ran on one macOS computer over the course's `cable` program, which sets C, T_{prop} and the bit error rate (BER), with bit errors in both directions. Each transfer sent `penguin.gif` (10 968 bytes) with our unmodified code. For the L tests, copies of the source used a different data chunk size; L is the size of the packet carried by each I frame: a 3-byte header plus L − 3 file bytes. We varied FER in two ways, as Table 1 shows: with the cable's bit errors, and with errors injected at the receiver, as the course guidelines suggest. For injected errors, a measurement copy of the receiver answers a correct new I frame with REJ with probability p, on a noise-free link.

Measurements. The transmitter times each transfer from the UA that opens the link to the start of the disconnection, so the time T covers the START packet, the data and the END packet. From T we compute R = 8 × (file bytes) / T and S = R / C, and we take the measured FER as retransmissions / (frames accepted + retransmissions). Each point is the mean of the runs in Table 1, and every received file was compared with the original. The baseline is C = 9600 bit/s, T_{prop} = 0, BER = 0 and L = 1000 bytes.

Table 1. Experiments. Parameters not listed keep the baseline value.

Parameter	Values	Runs
C (bit/s)	1200, 2400, 4800, 9600, 19 200, 38 400, 57 600, 115 200	3 × 8
T_{prop} (ms)	0, 10, 50, 100, 200, 500	3 × 6
FER, cable noise	targets 0, 0.1, 0.2, 0.3, 0.4, 0.5; BER from Equation 1	15 × 6
FER, injected REJ	p = 0, 0.1, 0.2, 0.3, 0.4, 0.5	5 × 6
L (bytes)	125, 250, 500, 1000, at T_{prop} = 0, at T_{prop} = 100 ms and at BER = $2.79 \cdot 10^{-5}$	3 × 12

Corrected model. Equation 1 treats every bit on the line as useful. On our link each byte takes 10 bit times (8-N-1), and each transfer also carries headers, stuffing, control packets and one 5-byte reply per frame. With n_i bytes on the line for frame i, counted from the actual file, the error-free time and the efficiency are

$$T_0 = \sum_i \left[\frac{10(n_i + 5)}{C} + 2T_{\text{prop}} \right], \quad S_c = \frac{8F}{C\,T_0}(1 - \text{FER}) \tag{2}$$

where F is the file size in bytes. With noise, each REJ adds one frame cycle $T_c = 10(n + 5)/C$ and each timeout adds 4 s to T_0.

Results

Figure 1 shows the measured efficiency with both models, and Table 2 gives the differences. All 198 transfers reached the end of the file, and 188 delivered an identical copy.

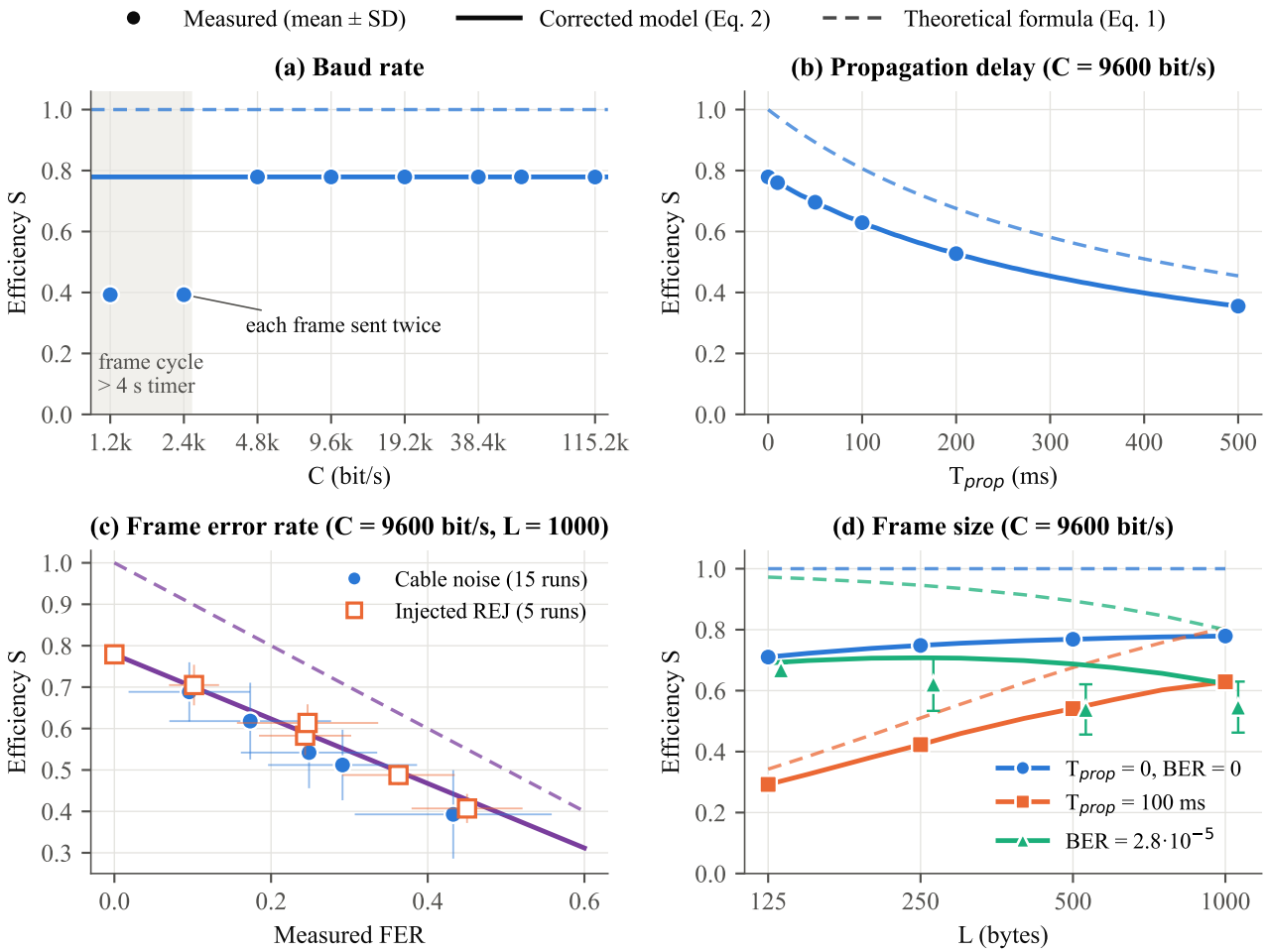


Figure 1. Measured efficiency S against (a) C, (b) T_{prop}, (c) measured FER and (d) L, with Equation 1 (theoretical) and Equation 2 (corrected). Colour marks the test condition; solid lines show Equation 2 and dashed lines Equation 1. Error bars show the standard deviation of the repetitions, and the shaded band in (a) marks the rates where a frame cycle exceeds the 4 s timer. In (c), whose y-axis starts at 0.25, both series share the purple model curves; in (d), the BER curves use the FER predicted by Equation 1, and the BER points are shifted right by 6% for legibility.

Table 2. Mean absolute difference between measured S and each model. The C row excludes 1200 and 2400 bit/s.

Experiment	Equation 1	Equation 2
C (T_{prop} = 0)	0.221	< 0.001
T_{prop}	0.177	< 0.001
L, T_{prop} = 0 / 100 ms	0.248 / 0.112	< 0.001 / < 0.001
FER, cable noise (using measured FER)	0.204	0.029
FER, injected REJ (using measured FER)	0.170	0.011
L, BER = $2.79 \cdot 10^{-5}$ (using measured FER)	0.259	0.045

Baseline. The error-free baseline takes T = 11.73 s, so R = 7479 bit/s and S = 0.779. The transfer puts 11 262 bytes on the line for 10 968 file bytes, which caps S at 8 × 10 968 / (10 × 11 262) = 0.779: the protocol reaches this limit, and the 10-bit byte alone caps S at 0.8.

Baud rate. S stays at 0.779 from 4800 to 115 200 bit/s, as $a \approx 0$ predicts. At 2400 and 1200 bit/s it halves to 0.39, because the largest frame and its reply take 4.27 s and 8.54 s, longer than the 4 s timer. The transmitter then resends frames that are still arriving: these runs show 12 to 13 retransmissions and the same number of duplicates at the receiver. With this timer, 1000-byte frames need C above about 2560 bit/s.

Propagation delay. S falls as T_{prop} grows, to 0.355 at 500 ms ($a = 0.6$), because the transmitter stays idle for each round trip. Equation 1 has the right shape but stays 1.28 times above every measurement, while Equation 2 matches all six points.

Frame error rate. S falls linearly with FER for both error sources, as both equations predict (Table 3). With injected REJs, every error costs exactly one frame cycle, and S follows Equation 2 to within 0.027 at every point. With cable noise, S lies 0.016 to 0.049 below Equation 2: noise hits frames in proportion to their length, so a full frame is 40 to 100 times as likely to be hit as one of the 3 short frames, and each of its retransmissions costs a 1.06 s cycle. The FER still counts the short frames, which injected errors hit as often as the full ones. Timeouts occurred at most 0.2 times per transfer with noise, and each costs 4 s. Counting REJs and timeouts reproduces T in all 87 noisy runs to within 0.18 s. With only 14 frames per file, S varies between runs, which is why the noise series uses 15 runs per point.

Table 3. Efficiency against FER (C = 9600 bit/s, $T_{prop} = 0$, L = 1000 bytes). Subscript n: cable noise, 15 runs per point; subscript i: REJs injected at the receiver with probability p equal to the target FER, 5 runs per point. SD is the standard deviation of S, and S_c is Equation 2 at the measured FER.

Target	BER	FER_n	S_n	SD	S_c	Corrupted	FER_i	S_i	SD	S_c
0	0	0	0.779	0.000	0.779	0	0	0.779	0.000	0.779
0.1	$1.32 \cdot 10^{-5}$	0.096	0.689	0.071	0.705	0	0.102	0.705	0.049	0.700
0.2	$2.79 \cdot 10^{-5}$	0.173	0.618	0.093	0.644	1	0.243	0.583	0.047	0.589
0.3	$4.46 \cdot 10^{-5}$	0.249	0.542	0.086	0.586	1	0.247	0.614	0.045	0.587
0.4	$6.39 \cdot 10^{-5}$	0.291	0.512	0.085	0.552	2	0.363	0.488	0.025	0.497
0.5	$8.66 \cdot 10^{-5}$	0.433	0.393	0.107	0.442	5	0.450	0.407	0.035	0.428

Frame size. Without errors, longer frames raise S from 0.710 at L = 125 to 0.779 at L = 1000, because overhead takes a smaller share, while Equation 1 gives S = 1 for every L. With $T_{prop} = 100$ ms the gain is larger, from 0.292 to 0.629, because a falls as L grows. With BER = $2.79 \cdot 10^{-5}$ the order reverses: the measured FER rises from 0.054 to 0.254, and L = 125 reaches S = 0.668 against 0.546 at L = 1000. The best L therefore depends on the link: 1000 bytes on a link without errors, and the shortest tested frame at this BER.

Undetected errors. Cable noise corrupted 9 of the 75 files received in the noisy FER runs, 5 of them at the highest BER; their time still counts in S. Replaying one showed two flipped bits in the same bit position of the same frame. BCC2 is a parity check per bit position, and parity misses any even number of errors [1, slide 20]. Injected REJs alter no data, and all 30 of those files arrived intact. The cable's noise generator is unseeded: repeating the FER sweep reproduced all 30 earlier runs exactly, and the three L = 1000 runs of the noisy L series repeat FER runs, which leaves 84 distinct noisy transfers.

Conclusions

The measurements confirm the behaviour that Equation 1 describes: S falls linearly with FER and as $1/(1 + 2a)$ with delay. On this link the formula overestimates S by a factor of 1.28, and counting start and stop bits, frame overhead and replies (Equation 2) brings the mean error below 0.001 on error-free points and to 0.011 with injected errors. Among the parameters varied alone from the baseline, T_{prop} lowered S the most, to 0.355 at 500 ms. A timer that scales with the frame time would remove the loss at 2400 bit/s and below, and a CRC in place of the XOR would catch the errors that reached the file. These results describe the virtual cable, where errors are independent and the delay is exact.

References

[1] M. P. Ricardo and R. Prior, "The Data Link Layer", Redes de Computadores lecture slides, Universidade do Porto, 2026/2027.